Assignment 2
Numerical Computations
and a Financial Calculator

by Kerry Veenstra
CSE 13S, Fall 2026
Document version 1

Due Wednesday, October 21st, 2026, at 11:30 pm

> You will be writing a C program that reads data from the keyboard (or from an input file), computes numeric values using given equations, and then prints numeric results to the screen (or to an output file).

Contents

1 This Assignment: What Will You Be Doing? **2**

2 Initial Preparation — Copy the Resource Files **2**

3 Running the Reference Program (and Financial Transactions) **3**
3.1 Background . . . 3
3.2 Examples . . . 4

4 A Financial Calculator **7**
4.1 Functional Specification . . . 7
4.2 Error Messages . . . 8

5 Functions That You Will Use **8**

6 Equations **9**
6.1 Equations for `PV`, `PMT`, and `FV` . . . 9
6.2 Computing N Numerically . . . 10
6.3 Computing I Numerically . . . 11

7 Your Task **12**
7.1 Design Your Program . . . 12
7.2 Write Your Program . . . 12
7.3 Submit Your Program . . . 15

8 Revisions **15**

A Appendix: Equation Details **15**

1 This Assignment: What Will You Be Doing?

This assignment serves two purposes:

1. Introduce you to—and let you practice using—**several features of the C programming language** and its **libraries**.
2. (Optional) Let you explore the **Time Value of Money**, a very useful concept in life and business.

The C features will include:

- ☐ Declaring and using *variables* and their *types*: `double`, `int`, and `char`; and a more complicated one, `char *`. You also will need to understand the difference between **global** and **local** variables. Unlike Python, C does *not* have a `global` keyword; the context determines the kind (*scope*) of the variable that you are declaring.
- ☐ Create *assignment statements* (like `x = 3;`) that assign a new value to a variable. More complex assignments include an *expression* as in the assignment `y = m * x + b;`.
- ☐ `if` / `else if` / `else` statements such as `if (b < 0) b = -b;`
- ☐ `for` statements such as `for (j = 0; j < 10; ++j) x = x + j;`
- ☐ Using the "character-string comparison" function `strcmp()`.
- ☐ *Defining* functions, *calling* functions, and and *returning values* from functions.
- ☐ Using the `fprintf()` function to print data from a variable to the screen (or to an output file), and using the `sscanf()` function to "scan" data from the keyboard (or from an input file) into a variable.
- ☐ Using the mathematical functions: `ceil(x)`, `fabs(x)`, `log(x)`, and `pow(x,n)`.

2 Initial Preparation — Copy the Resource Files

☐ **To do:** Below are the steps for synchronizing the **resources** repository on your Github Codespaces instance and then copying the files for asgn2 into your repository. Remember that the you need to use the `cd` ("change directory") command to set the "current directory" of the Terminal. **The Explorer window does not affect the current directory of the Terminal.**

Note in the first `cd` command below, you want to go to `/workspaces/codespaces-blank` (notice that this "absolute path" starts with a slash `/`), but you can use "auto completion" to reduce the amount of typing that you need to do. That is, instead of typing

```
$ cd /workspaces/codespaces-blank ⏎
```

You can type the first few letters of a directory name followed by a tab key, as demonstrated below.

```
$ cd /w tab /c tab ⏎
$ cd 13s/resources
$ git pull
$ cd ..
$ cp resources/asgn2/* yourcruzid/asgn2
$ cd yourcruzid/asgn2
$ ls
```

In the Terminal (or in the Explorer) you should see three executable files (also called "reference binaries" or "demo programs"): `tvm_arm`, `tvm_hub`, and `tvm_x86`. You will be running one of these binaries: `tvm_hub` (assuming that you are using GitHub). Then, after you make your own version of the program, you'll have one more binary: `tvm`.

3 Running the Reference Program (and Financial Transactions)

In this section you will run the reference program `tvm_hub`. (If you are not using GitHub, then you will use either `tvm_arm` or `tvm_x86`. The rest of this document will continue to instruct you to run `tvm_hub`, but if you are not using GitHub, you should use one of the other binaries instead.)

Let's see how the `tvm_hub` program works, starting with a quick introduction to the Time Value of Money.

3.1 Background

3.1.1 Time Value of Money

Let's start with two questions: If you lent someone $10,000, would you want to be paid back $10,000 now or $10,000 later? If someone lent *you* $10,000, would you want to pay *them* back $10,000 now or $10,000 later? Most people would prefer to *get* money *sooner* but *pay* it *later*.

Consider a similar but more *quantitative* question: if you put $10,000 in a bank for one year, how much money would you want the bank to pay you back one year from now? More than $10,000, right? Is it $10,100? $10,200? $10,300? Let's say it's $10,300. That means that you consider 10,000 current dollars and 10,300 future dollars to be equally valuable, demonstrating the "time value of money" (TVM). That is, a dollar in the future is worth less than a dollar now, and so to get the same value that you get with 10,000 "current dollars," you would need 10,300 of the less-valuable "future dollars."

One can relate these two values mathematically with an interest rate, `I`. In our example, the interest rate is 3% (that is, in this equation $\texttt{I} = 0.03$). [1]

$$\text{future dollars} = \text{current dollars} \times (1 + \texttt{I})$$

3.1.2 Cash-flow Diagrams

The previous section talked about *value*, but modern financial calculators consider *signed cash flows* instead. That is, using the modern approach, from your point of view, the savings deposit above is characterized by a *negative cash flow* (a "cash outflow") representing the money that you *paid out*. The savings withdrawal one year later is characterized by a *positive cash flow* (a "cash inflow") representing the money that you *received*.

Cash flows are given different names depending on when they happen. A cash flow at the beginning of the analysis is called the "Present Value," or `PV`. A cash flow at the end of the analysis is called the "Future Value," or `FV`. (Equal, periodic cash flows in the middle are called "Payments," whose values are represented by `PMT`. In this example $\texttt{PMT} = 0$.) To summarize:

- Present value $\texttt{PV} = -\$10{,}000$ means that you have a cash outflow of $10,000 (negative for money that you pay out—in this case to the bank).
- Future value $\texttt{FV} = \$10{,}300$ means that one year later you have a cash inflow of $10,300 (positive for money that you receive—in this case from the bank).
- Number of periods $\texttt{N} = 1$ means that the interest is computed after one "period" (year).
- Interest rate $\texttt{I} = 0.03$ means that the interest rate is 3% per period (year). Notice that `I` and `N` must use the same period.
- Payment $\texttt{PMT} = 0$ means that in this example there are no payments.

We represent these transactions using the cash-flow diagram in Fig 2 . The Figs. 1a and 1b explain the diagram's notation. (References [1] and [2] are two of many sources that describe uses of cash-flow diagrams.)

[1] A quick note on notation: this assignment uses upper-case letters for all financial variables to avoid a confusing name for a global variable in the program. If you learn about—or already know about—finance, you will see i and n in your finance book instead of the `I` and `N` that are used in this assignment.

Figure 1: Notation used in cash-flow diagrams.

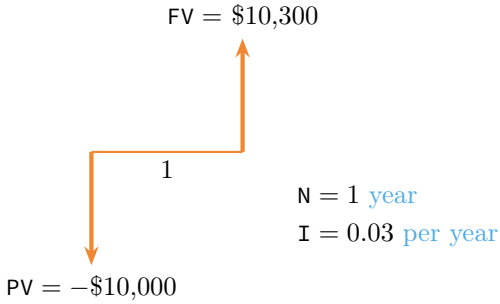
(a) A time line is divided into compounding periods. This time line is divided into six compounding periods.



(b) On a time line, vertical arrows indicate the exchange of money. An upward arrow indicates money received and a downward arrow indicates money paid out.



Figure 2: A cash-flow diagram of a simple savings account with an annual interest rate of 3% (`I` = 0.03). Interest is credited once, at the end of the year.



3.2 Examples

3.2.1 Simple Interest

Let's do a financial calculation. The values of Fig. 2 are related using this simplified Time Value of Money (TVM) equation:

$$\texttt{FV} = -\texttt{PV} \times (1 + \texttt{I})^{\texttt{N}} \tag{1}$$

We can use equation (1) to compute the value of `FV` manually from `N`, `I`, and `PV`:

$$\begin{aligned} \texttt{FV} &= -\texttt{PV} \times (1 + \texttt{I})^{\texttt{N}} \\ &= -(-10000) \times (1 + 0.03)^{1} \\ &= 10000 \times 1.03 \\ &= 10300 \end{aligned}$$

Your `tvm` program will be able to do the same computation. Fig. 3a is the listing of an input file `simple.dat` that describes the cash-flow diagram of Fig. 2. Fig. 3b shows the Linux command needed to use `simple.dat` as input to `tvm_hub` to compute `FV`. The result is `FV` = $10,300.00.

☐ **To do:** Find the input file `simple.dat` (Fig. 3a). Verify its contents—either using the Explorer in Github or the `cat` command in the Terminal.

☐ **To do:** In the Terminal, process `simple.dat` (Fig. 3b).
☐ **To do:** Confirm that the result is `FV` = 10300.00.

Figure 3: Using `tvm_hub` with simple interest.

```
set N 1
set I 0.03
set PV -10000
compute FV
```

(a) File `simple.dat`

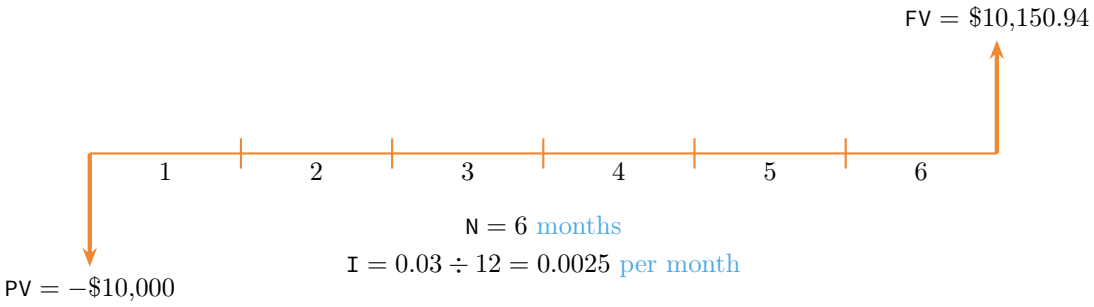
```
$ ./tvm_hub < simple.dat
FV = 10300.00
$
```

(b) Running `tvm_hub`

3.2.2 Compound Interest

Explore *compound interest* using the explanations in this section.

Figure 4: Cash-flow diagram of a savings account with an annual interest rate of 3% but whose interest is compounded monthly. Typically in finance, given an *annual* interest rate, you compute the interest rate per month by dividing by 12 (the number of months in a year). Here, the interest rate per month is `I` = 0.03 ÷ 12 = 0.0025.



In this case, the total duration of the analysis is only half a year, or six months, which suggests that we will get half of the $300 of interest of the simple example, or $150. However, comparing the values of `PV` and `FV`, (in Fig. 4) you see that you are getting $150.94, and so compounding (collecting interest on the bank account's increasing balance), results in $0.94 more interest.
☐ **To do:** Find the input file `compound1.dat` (Fig. 5a). Verify its contents—either using the Explorer in Github or the `cat` command in the Terminal.
☐ **To do:** In the Terminal, process `compound1.dat` (Fig. 5b).
☐ **To do:** Confirm that the result is `FV` = 10150.94.

Figure 5: Computing `FV` with compound interest.

```
set N 6
set I 0.0025
set PV -10000
compute FV
```

(a) File `compound1.dat`

```
$ ./tvm_hub < compound1.dat
FV = 10150.94
$
```

(b) Running `tvm_hub`

3.2.3 Computing I

Explore an *interest calculation* using the explanations in this section.

The power of a financial calculator like tvm is that it can compute the *other* financial values, such as I. Let's say that we start with the prior analysis (Fig. 4), but that you want FV =$10,200 instead of $10,150.94. What interest rate is needed to get that FV? Figs. 6a and 6b show how to compute this interest rate using tvm_hub and the input file compound2.dat.

☐ **To do:** Create the input file compound2.dat, as shown in Fig. 6a.

☐ **To do:** Process the input file using tvm_hub, as shown in Fig. 6b.

The result, I = 0.003306 is a *monthly* interest rate, and so you need to multiply it by 12 to get an *annual* interest rate: $0.003306 \times 12 = 0.0397$, which is 3.97% (more than the original 3%).

Figure 6: Computing I for a cash flow with compound interest.

```
set N 6
set PV -10000
set FV 10200
compute I
```

(a) File compound2.dat

```
$ ./tvm_hub < compound2.dat
I = 0.003306
$
```

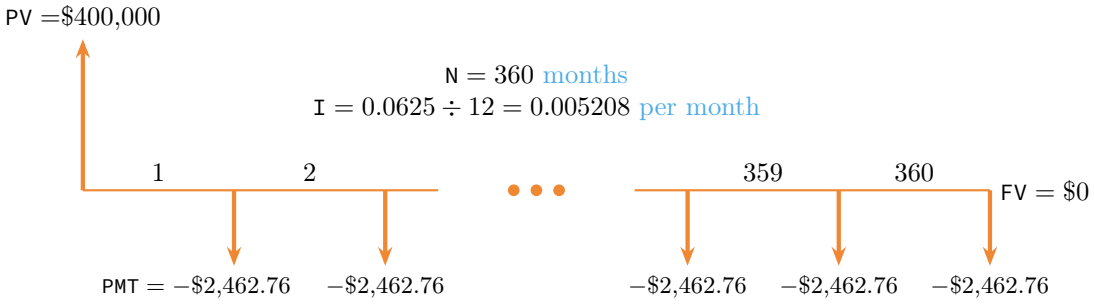
(b) Running tvm_hub

3.2.4 30-year Mortgage

Explore a *30-year mortgage* using the explanations in this section.

Let's consider payments, as one would have in a mortgage loan. You see that the cash-flow diagram in Fig. 7 has more arrows: The upward arrow represents getting cash from the original loan, the 360 downward arrows (not all are shown) represent 360 monthly mortgage payments over 30 years, and the zero-value FV (which is what one gets with a fully "amortized" loan) has no arrow, but is mentioned at the end of the time line. (By the way, getting a mortgage does not mean that you are stuck in the same place for 30 years: if you sell the condo, you can pay off the loan early and buy a new place with a new loan.)

Figure 7: Cash-flow diagram of a 30-year mortgage with an annual interest rate of 6.25%. The bank gives you $400,000 to help you buy a condominium, and in exchange you agree to make 360 monthly payments of $2,462.76. After the 360th payment, the loan balance becomes 0, and you have finished paying off the loan.



Figs. 8a and 8b show how to compute a payment PMT using tvm_hub and the input file mortgage1.dat. The result is PMT = −2462.76.

☐ **To do:** Create the input file mortgage1.dat, as shown in Fig. 8a.

☐ **To do:** Process the input file using `tvm_hub`, as shown in Fig. 8b. Does the `tvm_hub` program report a payment of $2,462.76?

Figure 8: Using `tvm_hub` to compute a mortgage payment.

```
set N 360
set I 0.005208
set PV 400000
set FV 0
compute PMT
```

(a) File `mortgage1.dat`

```
$ ./tvm_hub < mortgage1.dat
PMT = -2462.76
$
```

(b) Running `tvm_hub`

Finally, let's say that you are more comfortable with a $2,300 monthly payment (`PMT`). What size of loan (`PV`) would that payment support?
☐ **To do:** Create the input file `mortgage2.dat`, as shown in Fig. 9a.
☐ **To do:** Process the input file using `tvm_hub` as shown in Fig. 9b. Does the `tvm_hub` program report a present value of $373,563.90?

Figure 9: Using `tvm_hub` to compute a mortgage `PV`.

```
set N 360
set I 0.005208
set PMT -2300
set FV 0
compute PV
```

(a) File `mortgage2.dat`

```
$ ./tvm_hub < mortgage2.dat
PV = 373563.90
$
```

(b) Running `tvm_hub`

4 A Financial Calculator

The prior section demonstrated a number of uses for a financial calculator. This section provides the specification.

4.1 Functional Specification

Here is the specification for your `tvm` program:

- The program is called `tvm`.
- You can run the program as one of:
  - `./tvm`
  - `./tvm <` *filename*
- The program maintains the values of five financial variables: `N`, `I`, `PV`, `PMT`, and `FV`.
- The initial value of all of the financial variables is 0.
- The input data is a sequence of commands. The list below defines all of the commands that need to be supported. Within the input file, the commands can be in any order, but you need to *set* values before computing other values that depend on them. All of the italicized terms represent floating-point numbers.

```
set N number_of_periods     ⟵ set the value of N (N must be a positive integer)
set I interest_rate         ⟵ set the value of I (I must be positive)
set PV present_value        ⟵ set the value of PV
set PMT payment             ⟵ set the value of PMT
set FV future_value         ⟵ set the value of FV

compute N                   ⟵ compute N and print it to 0 decimal positions: %.0f
compute I                   ⟵ compute I and print it to 6 decimal positions: %.6f
compute PV                  ⟵ compute PV and print it to 2 decimal positions: %.2f
compute PMT                 ⟵ compute PMT and print it to 2 decimal positions: %.2f
compute FV                  ⟵ compute FV and print it to 2 decimal positions: %.2f

clear                       ⟵ clear all financial variables (N, I, PV, PMT, and FV) to 0
test                        ⟵ run simple tests of four of your functions
blank line                  ⟵ ignore a blank line
```

- An input file can define more than one computation by including additional `set` and `compute` commands. Also, you can use the `clear` command as a shortcut to setting all of the financial variables to 0.
- If you run the command as just `./tvm`, then it will take input from the keyboard. To end the input use ctrl + D.

4.2 Error Messages

Your `tvm` program should check for the following error conditions. **If one of the conditions is met, then print the error message, and call `exit(1)`.** The error messages are shown as `printf()` format strings. The `%d` is used to print the line number of the input line that has the error. (The colored part of the table is used in an example, below.)

Command	Error Condition	Error Message to Print
`set` *name*	*name* is not a valid variable name	`line %d: invalid variable name\n`
`set N` *value*	*value* is not a positive integer	`line %d: N must be a positive integer\n`
`set I` *value*	*value* is not positive	`line %d: I must be positive\n`
`compute` *name*	*name* is not a valid variable name	`line %d: invalid variable name\n`
`compute` *name*	*name* is not `I`, and $I = 0$	`line %d: I must not be zero\n`
`compute N`	after loop: $\left\lvert \frac{f(N_k)}{f'(N_k)} \right\rvert \geq 10^{-8}$	`line %d: solver did not converge\n`
`compute I`	after loop: $\left\lvert \frac{f(I_k)}{f'(I_k)} \right\rvert \geq 10^{-8}$	`line %d: solver did not converge\n`
other	unknown command	`line %d: invalid command\n`

The character strings in the rightmost column of the table above are to be used in a call to the `fprintf()` function. Here is an example:

```
fprintf(stderr, "line %d:  invalid variable name\n", line_number);
```

Notice that I've added a `\n` after each error message. Be sure to include it!

5 Functions That You Will Use

Here is a list of some of the standard functions that you will use. (Use the Unix/Linux `man` command for more information. Remember that pressing q exits the `man` command and that you can jump straight to the RETURN VALUE section by searching for RETURN with / + R + E + T + U + R + N + ⏎.)

- `sscanf()` — Recognizes a command and parses a command string into "words." Returns the number of `%` fields that match the given input string. Hint: to check if a string is a `set` command, use the `sscanf()` format string `"set %s %lf"`, and if that fails to match, then check for the `compute` command by using the format string `"compute %s"`. If neither, then use `strcmp` to check for `"clear"`, and if that fails, then check for `""`.
- `ceil(x)` — Rounds a `double` value *up* to the closest integral value. Examples: `ceil(1.3)` → `2.0` but `ceil(2.0)` → `2.0`. Use in equation (10). Also, you can use `ceil()` when checking whether `N` is a positive integer since `x == ceil(x)` is true only when `x` has an integral value.
- `fabs(x)` — Returns the *absolute value* of the `double` value `x`. Use when determining that the Newton-Raphson loop is done. See inequality (5).
- `log(x)` — Computes $\ln x$. (Yes, ln, or "natural log." The function's name is misleading.) Use in equation (8).
- `pow(x,n)` — Computes x^n. Use to compute $(1+I)^N$ and $(1+I)^{-N}$ in equations (2), (3), (4), (7), (8), (12), and (13).

6 Equations

This section lists the equations that you will use. (If you are curious, see the Appendix for their derivation.)

Here are the variables that the equations use. The signs of the variables `PV`, `PMT`, and `FV` are positive for money received and negative for money paid:

`N` = "Number of Periods": the number of months or the number of years.

`I` = "Interest Rate": interest per month or interest per year; period must match that of `N`.

`PV` = "Present Value": the value of the *first transaction*.

`PMT` = "Payment": the value of each regular, *periodic payment*.

`FV` = "Future Value": the value of the *last transaction*.

6.1 Equations for `PV`, `PMT`, and `FV`

$$\mathtt{PV} = -\mathtt{PMT} \times \frac{1-(1+\mathtt{I})^{-\mathtt{N}}}{\mathtt{I}} - \mathtt{FV} \times (1+\mathtt{I})^{-\mathtt{N}} \tag{2}$$

$$\mathtt{PMT} = \mathtt{I} \times \frac{\mathtt{PV} \times (1+\mathtt{I})^{\mathtt{N}} + \mathtt{FV}}{1-(1+\mathtt{I})^{\mathtt{N}}} \tag{3}$$

$$\mathtt{FV} = -\mathtt{PV} \times (1+\mathtt{I})^{\mathtt{N}} - \mathtt{PMT} \times \frac{(1+\mathtt{I})^{\mathtt{N}}-1}{\mathtt{I}} \tag{4}$$

You will need to represent these equations in the C programming language. Here is how (2) would be represented in C:

```
PV = -PMT * (1 - pow(1 + I, -N)) / I - FV * pow(1 + I, -N);
```

One who notices that the subexpression $(1 + \texttt{I})^{-\texttt{N}}$ = `pow(1 + I, -N)` appears twice in the C assignment statement might compute this subexpression once and assign it to an intermediate variable (with a name of their choosing) for use in the equation.

One also might notice that equations (3) and (4) share the subexpression $(1 + \texttt{I})^{\texttt{N}}$ = `pow(1 + I, N)`, and so one might compute this *second* subexpression once and assign it to a different intermediate variable for use in *those* equations.

Note that there is one limitation that you must consider. Look carefully at equations (2), (3), and (4): they will divide by zero when $\texttt{I} = 0$. **So when computing the value of `N`, `PV`, `PMT`, or `FV`, if $\texttt{I} = 0$, then report an error (section 4.2), and exit the program.** (Although these equations have defined limits when $\texttt{I} \to 0$, we aren't going to complicate the program with that special case.)

6.2 Computing `N` Numerically

You cannot use algebra to solve for either `N` or `I`, but you can use numerical methods. For this assignment, you will use the *Newton-Raphson* method. Here is the idea when solving for `N`:

1. Define a function $f(\texttt{N})$ that will equal 0 when you have found the target value of `N`. That is, the algorithm will strive to find a value for `N` such that $f(\texttt{N}) = 0$.
2. Have a loop generate a sequence of values $\texttt{N}_0, \texttt{N}_1, \texttt{N}_2, \ldots$ that make $f(\texttt{N}_k) \to 0$. (See equations (7), (8), and (9).)
3. Stop when the fraction $\frac{f(\texttt{N}_k)}{f'(\texttt{N}_k)}$ of equation (9) is sufficiently close to 0. At that point, $\texttt{N}_k$ is your desired solution. We are going to define "sufficiently close to 0" as

$$\left| \frac{f(\texttt{N}_k)}{f'(\texttt{N}_k)} \right| < 10^{-8} \tag{5}$$

   Also, to avoid an infinite loop, stop after the loop generates $\texttt{N}_{10000}$ (it probably won't).

Here are equations for using the Newton-Raphson method to solve for `N`. To simplify this assignment, we are using a constant initial value $\texttt{N}_0 = 360$.[2]

$$\texttt{N}_0 = 360 \tag{6}$$

$$f_{\texttt{N}}(\texttt{N}_k) = \texttt{PV} + \texttt{PMT} \times \left[\frac{1 - (1 + \texttt{I})^{-\texttt{N}_k}}{\texttt{I}} \right] + \texttt{FV} \times (1 + \texttt{I})^{-\texttt{N}_k} \tag{7}$$

$$f'_{\texttt{N}}(\texttt{N}_k) = \ln(1 + \texttt{I}) \times \left[\texttt{PV} + \frac{\texttt{PMT}}{\texttt{I}} \right] \times (1 + \texttt{I})^{\texttt{N}_k} \tag{8}$$

$$\texttt{N}_{k+1} = \texttt{N}_k - \frac{f_{\texttt{N}}(\texttt{N}_k)}{f'_{\texttt{N}}(\texttt{N}_k)} \tag{9}$$

$$\texttt{N} = \texttt{ceil}(\texttt{N}_{\text{final}}) \tag{10}$$

[2] Typically when using the Newton-Raphson method, one's choice of initial values depends on the values of other variables. For simple, single-variable equations, one typically uses a table, with the idea being to choose starting values that reduce the number of Newton-Raphson iterations. When using the Newton-Raphson method for financial calculations of $\texttt{N}_0$, more care is needed than we are taking in this assignment[3].

Here is the pseudocode for using the Newton-Raphson method to solve for N.

```
def newton_raphson_N():
    # initial value from equation (6)
    N = 360

    # we need the value of delta after the "for" loop
    delta = 0

    for k in range (0, 10001):
        # equations (7), (8), and (9)
        delta = - fN(N) / fN_prime(N)
        N = N + delta

        if fabs(delta) < 1e-8:
            break;

    if fabs(delta) >= 1e-8:
        report error "solver did not converge"
        exit program with return code of 1

    # equation (10)
    return ceil(N)
```

6.3 Computing I Numerically

Here are the equations for using the Newton-Raphson method to solve for I. To simplify the assignment, we are using a constant initial value $\mathtt{I}_0 = 0.0025$.

$$\mathtt{I}_0 = 0.0025 \tag{11}$$

$$f_{\mathtt{I}}(\mathtt{I}_k) = \mathtt{PV} + \mathtt{PMT} \times \left[\frac{1 - (1 + \mathtt{I}_k)^{-\mathtt{N}}}{\mathtt{I}_k}\right] + \mathtt{FV} \times (1 + \mathtt{I}_k)^{-\mathtt{N}} \tag{12}$$

$$f'_{\mathtt{I}}(\mathtt{I}_k) = \mathtt{N} \times \left[\mathtt{PV} + \frac{\mathtt{PMT}}{\mathtt{I}_k}\right] \times (1 + \mathtt{I}_k)^{\mathtt{N}-1} - \mathtt{PMT} \times \frac{(1 + \mathtt{I}_k)^{\mathtt{N}} + 1}{\mathtt{I}_k^2} \tag{13}$$

$$\mathtt{I}_{k+1} = \mathtt{I}_k - \frac{f_{\mathtt{I}}(\mathtt{I}_k)}{f'_{\mathtt{I}}(\mathtt{I}_k)} \tag{14}$$

$$\mathtt{I} = \mathtt{I}_{\text{final}} \tag{15}$$

Here is the pseudocode for using the Newton-Raphson method to solve for `I`.

```
def newton_raphson_I():
    # initial value from equation (11)
    I = 0.0025

    # we need the value of delta after the "for" loop
    delta = 0

    for k in range (0, 10001):
        # equations (12), (13), and (14)
        delta = - fI(I) / fI_prime(I)
        I = I + delta

        if fabs(delta) < 1e-8:
            break;

    if fabs(delta) >= 1e-8:
        report error "solver did not converge"
        exit program with return code of 1

    # equation (15)
    return I
```

7 Your Task

7.1 Design Your Program

This program can be created as a single C file.
☐ **To do:** While reading Sections 3–6, take notes on what your program, `tvm.c`, needs to do.
☐ **To do:** If necessary, create a few small test programs to confirm your understanding of how to use the functions in section 5.
☐ **To do:** Based on your notes, create a program design in pseudocode.

7.2 Write Your Program

7.2.1 Starter Code

We've given you starter code in the file `tvm.c`. You'll see that each of the functions has a "header comment" that describes the function's purpose, parameters, and return value, but you'll also see that we've left the bodies of most of the functions empty.

7.2.2 Start with One Function: `tvm_process_command()`

☐ **To do: Edit:** To start, write the function body for `tvm_process_command()`. Your function will use `sscanf()` and `strcmp()` to determine which command is represented in the string parameter `command`. (Section 5 has a hint about this.) Depending on what's in the parameter `command`, your function body will call `tvm_set_variable()`, `tvm_compute_variable()`, or `tvm_clear()`. (Or ignore a blank command.)
☐ **To do: Compile:** Although you've written only one function at this point, you can check it by compiling your program using the command below. (The `-lm` compiler option is needed because we are using functions from the math library: `ceil()`, `fabs()`, `log()`, and `pow()`. This is the only library that we are going to use in this class that requires a special command-line compiler option.)

```
$ clang -lm tvm.c -o tvm
$
```

Were there errors? Although error messages usually are verbose, the first thing printed probably will give a line number in `tvm.c`. That's the place to look first. Repeat these first two steps until your program compiles without errors.

By the way, if you forget the `-lm` command-line option, then you will get eleven `undefined reference` errors and a `linker command` error. Re-run the `clang` command as shown above, but remember to include the `-lm` command-line option!

At this point, your program doesn't do anything useful, but you *can* run it and confirm that it can read a file.

☐ **To do: Run:** Run your program:

```
$ ./tvm < simple.dat
$
```

It won't print anything useful, but we're about to change that.

7.2.3 Function Number Two: `tvm_set_variable()`

☐ **To do: Edit:** Write the function body for `tvm_set_variable()`. Your function will use `if/then/else` statements and `strcmp()` to determine which variable name has been provided. For each of the variable names `"N"`, `"I"`, `"PV"`, `"PMT"`, and `"FV"`, set the global variable of the same name to the `value` parameter. Also, your function needs to check the conditions for the first three error messages in the table of section 4.2. Report an error message like this:

```
    fprintf(stderr, "line %d:  message from the table\n", line_number);
    exit(1);
```

☐ **To do: Compile:** Recompile your program (using the `clang` command from earlier), fixing any errors.
☐ **To do: Run:** You can run your program and force it to generate error messages. Check these things: Can your program set the five variable names?

```
$ ./tvm
set N 12
set I 0.03
set PV 0
set PMT 0
set FV 0
ctrl+D        ← End of keyboard input:  while holding the ctrl key, press the D key.
$
```

Can your program detect each of the error conditions that it is supposed to report?

```
$ ./tvm
set badname 0
line 1:  invalid variable name
$
```

```
$ ./tvm
set N 0
line 1:  N must be a positive integer
$
```

```
$ ./tvm
set N 1.234
line 1:  N must be a positive integer
$
```

```
$ ./tvm
set I -0.03
line 1:  I must be positive
$
```

If necessary, fix any bugs. Is everything working? Then take a snapshot of your work to easily recover from a wayward file deletion or accidental major edit.
☐ **To do:** Periodically `git add`, `git commit`, and `git push` your `tvm.c` file. (See Assignment 0.)

```
$ git add tvm.c
$ git commit -m 'your own description of this version'
$ git push
$
```

7.2.4 Prepare to Run Unit Tests

We've included a function called `run_unit_tests` that does some minimal testing of your functions `fN()`, `fI()`, `fN_prime()`, and `fI_prime()`. (A "unit test" is a common name for a "function test.")
☐ **To do:** Edit `tvm_process_command()` and add a `test` command that calls `run_unit_tests()`.

7.2.5 Write the Rest of the Functions

For every empty function in `tvm.c`:

☐ **To do:** **Edit** the next function that has an empty body.
☐ **To do:** **Compile** the program with `clang -lm tvm.c -o tvm` to check for syntax errors.
☐ **To do:** **Run** the program with `./tvm` and use the `test` command that you added to run the unit tests. Exit the program with ctrl+D. Of course you can ignore any unit test of a function that you haven't yet completed.
☐ **To do:** **Run** the program with `./tvm` to confirm that your new features are working correctly.

7.3 Submit Your Program

There are three steps to submitting your program:

- `git add/commit/push` your final version of `tvm.c`.
- Run the Pipeline at `git.ucsc.edu` (see Assignment 0).
- Submit your final commit ID to Canvas by Wednesday, October 21st, 2026, at 11:30 pm.

You *may* run the Pipeline and submit a commit ID to Canvas as much as you want. So even if you have only some of the points, submit a commit ID for your work so far, in case something prevents you from finishing the program.

Also, we recommend creating and running your own local tests.

8 Revisions

Version 1 Original.

References

[1] Scott Besley and Eugene F. Brigham. *Essentials of Managerial Finance*, chapter 3: The Time Value of Money, pages 88–111. South-Western, Thomson, 13th edition, 2005.

[2] Hewlett-Packard, Chris Beal, Tan Tek Lum, Jacob Graudenz, Thomas Gauldin, and Howard Benedict. *The HP-92 Investor Financial Printing Calculator Owner's Handbook*, chapter 3: Financial Interest Calculations, pages 27–41. Hewlett-Packard Company, 1977. URL: `https://literature.hpcalc.org/community/hp92-oh-en.pdf`.

[3] Roy E. Martin. Printing financial calculator sets new standards for accuracy and capability. *Hewlett-Packard Journal*, 29(2):22–28, 1977. URL: `https://hparchive.com/Journals/HPJ-1977-10.pdf`.

A Appendix: Equation Details

This assignment uses the Time Value of Money equation (16), which handles situations where payments occur at the beginning of each period. (Equations of a real financial calculator are a bit more complicated and include a BEGIN/END setting for situations where payments occur at the end of each period.)

$$0 = \mathtt{PV} + \mathtt{PMT} \times \left[\frac{1-(1+\mathtt{I})^{-\mathtt{N}}}{\mathtt{I}}\right] + \mathtt{FV} \times (1+\mathtt{I})^{-\mathtt{N}} \tag{16}$$

We can use algebra to solve equation (16) for `PV`, `PMT`, and `FV`, deriving equations (2), (3), and (4). Solving for `N` and `I` numerically requires the partial derivatives of (7) and (12):

$$f'_{\mathtt{N}}(\mathtt{N}_k) = \frac{\partial f_{\mathtt{N}}}{\partial \mathtt{N}_k} = \ln(1+\mathtt{I}) \times \left[\mathtt{PV} + \frac{\mathtt{PMT}}{\mathtt{I}}\right] \times (1+\mathtt{I})^{\mathtt{N}_k} \tag{17}$$

$$f'_{\mathtt{I}}(\mathtt{I}_k) = \frac{\partial f_{\mathtt{I}}}{\partial \mathtt{I}_k} = \mathtt{N} \times \left[\mathtt{PV} + \frac{\mathtt{PMT}}{\mathtt{I}_k}\right] \times (1+\mathtt{I}_k)^{\mathtt{N}-1} - \mathtt{PMT} \times \frac{(1+\mathtt{I}_k)^{\mathtt{N}}+1}{\mathtt{I}_k^2} \tag{18}$$